PHASE 8 OF 18

Object Class, Wrappers, Math & Immutability

toString, equals and hashCode contracts, getClass, shallow vs deep cloning, wrapper classes, autoboxing and the Integer cache, the Math class, BigDecimal for money, and designing immutable classes

IN THIS PHASE

8.1 java.lang.Object: The Root of Every Class
8.2 toString()
8.3 equals(): Logical Equality
8.4 hashCode(): Must Agree with equals()
8.5 getClass() and a Glimpse of Reflection
8.6 clone(): Shallow vs Deep Copy
8.7 finalize() and Resource Cleanup
8.8 Wrapper Classes
8.9 The Math Class
8.10 BigDecimal and BigInteger
8.11 Immutable Classes
8.12 Predict the Output
8.13 Interview Questions
8.14 Quick Revision

8.1 java.lang.Object: The Root of Every Class

Every class directly or indirectly extends `java.lang.Object`. If you write `class User { }`, the compiler treats it as `class User extends Object { }`. So every object, including every array, has these **11 methods**:

Method	Returns	Can override?	Default behaviour	Used for
`toString()`	`String`	Yes	`ClassName@hexHashCode`	Printing, logging, debugging
`equals(Object o)`	`boolean`	Yes	Reference comparison (`this == o`)	Logical equality
`hashCode()`	`int`	Yes	Identity-based number (not an address)	`HashMap`, `HashSet` buckets
`getClass()`	`Class<?>`	No (`final`)	Runtime class of the object	Reflection, type checks
`clone()`	`Object`	Yes (`protected`)	Shallow field copy, if the class is `Cloneable`	Copying objects
`finalize()`	`void`	Yes	Nothing (**deprecated** since Java 9)	Avoid it
`wait()`, `wait(long)`, `wait(long, int)`	`void`	No (`final`)	Releases the monitor and waits	Inter-thread communication (Phase 11)
`notify()`, `notifyAll()`	`void`	No (`final`)	Wakes waiting thread(s)	Inter-thread communication (Phase 11)

8.2 toString()

`System.out.println(obj)` and string concatenation (`"x" + obj`) call `obj.toString()` automatically. The default output is the class name, `@`, and the hash code in hexadecimal, which is rarely useful.

```
class Student {
    int id = 101; String name = "Rohit";
}
System.out.println(new Student());     // Student@1b6d3586 (value varies per run)
```

```
class Student {
    private final int id;
    private final String name;
    Student(int id, String name) { this.id = id; this.name = name; }

    @Override
    public String toString() {
        return "Student{id=" + id + ", name='" + name + "'}";
    }
}
System.out.println(new Student(101, "Rohit"));   // Student{id=101, name='Rohit'}
```

`String`, the wrapper classes, `StringBuilder`, all collections, `LocalDate` and records already override `toString()`.

> **TIP**
> Let the IDE generate `toString()` (Eclipse: Source → Generate toString; IntelliJ: Alt+Insert), or use Lombok's `@ToString` or a record. **Never include passwords, card numbers or PINs** in `toString()`, because it ends up in logs.

8.3 equals(): Logical Equality

The default `Object.equals()` is `==`: it compares **references**. Override it when two different objects should count as **equal by content** (for example, two `Account` objects with the same account number).

The equals contract

Property	Meaning
Reflexive	`x.equals(x)` is `true`
Symmetric	`x.equals(y)` ⇔ `y.equals(x)`
Transitive	`x.equals(y)` and `y.equals(z)` ⇒ `x.equals(z)`
Consistent	Repeated calls give the same result while the objects are unchanged
Non-null	`x.equals(null)` is `false` (never throws)

A correct implementation

```
import java.util.Objects;

public final class Account {
    private final String accountNo;
    private final String ifsc;
    private double balance;            // not part of identity

    public Account(String accountNo, String ifsc) {
        this.accountNo = accountNo; this.ifsc = ifsc;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                          // 1. same reference
        if (!(o instanceof Account other)) return false;     // 2. type + null check (Java 16)
        return accountNo.equals(other.accountNo)             // 3. compare significant fields
            && Objects.equals(ifsc, other.ifsc);             //    null-safe for nullable fields
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNo, ifsc);                // SAME fields as equals()
    }
}
```

CORRECTED FROM ORIGINAL NOTES

The original notes' `equals` casts directly with `Student s = (Student) obj;`. That throws `ClassCastException` for other types and `NullPointerException` for `null`, violating the contract. Always check `this == o`, then the type and null with `instanceof` (or `getClass()`), before comparing fields.

NOTE

`instanceof` vs `getClass()`: `instanceof` lets subclass instances compare equal to parent instances, which can break symmetry if the subclass adds fields. `getClass() != o.getClass()` requires the exact same class. Making the class `final`, as above, avoids the debate.

8.4 hashCode(): Must Agree with equals()

Hash-based collections (`HashMap`, `HashSet`, `ConcurrentHashMap`) first use `hashCode()` to choose a **bucket**, and then use `equals()` to find the match inside that bucket.

The hashCode contract

1. Calling it repeatedly on an unchanged object returns the same value (within one run).
2. **If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` must be true.**
3. Unequal objects *may* share a hash code (a collision), but fewer collisions means better performance.

```
class Emp {                                    // overrides equals but NOT hashCode
    int id;
    Emp(int id) { this.id = id; }
    @Override public boolean equals(Object o) {
        return o instanceof Emp e && e.id == id;
    }
}

Set<Emp> set = new HashSet<>();
set.add(new Emp(1));
set.add(new Emp(1));
System.out.println(set.size());                  // 2  <- duplicate stored!
System.out.println(set.contains(new Emp(1)));    // false <- can't find an "equal" object
// Add hashCode() { return Integer.hashCode(id); }  -> size 1, contains true
```

REMEMBER

Always override `equals()` **and** `hashCode()` **together**, using the **same fields**. Equal objects must have equal hash codes; the reverse doesn't need to hold.

PITFALL

Don't mutate a field used in `hashCode()` while the object is inside a `HashSet` or used as a `HashMap` key. Its bucket no longer matches, so `contains`/`get` fail and `remove` silently does nothing. Prefer **immutable keys** (String, Integer, records).

	==	equals()	hashCode()
Compares	References (or primitive values)	Logical content (if overridden)	n/a: returns a number
Default	n/a	Same as ==	Identity-based
Overridden by	n/a	String, wrappers, collections, records, `LocalDate`, `BigDecimal`	Same classes

PITFALL

`BigDecimal.equals()` also compares **scale**: `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` is `false`, while `compareTo` returns `0`. Use `compareTo() == 0` for numeric equality of amounts.

8.5 getClass() and a Glimpse of Reflection

`getClass()` returns the `Class` object describing the object's **runtime** class. It is `final`, so it can't be overridden.

```
Object o = new ArrayList<String>();
Class<?> c = o.getClass();
System.out.println(c.getName());          // java.util.ArrayList
System.out.println(c.getSimpleName());    // ArrayList
System.out.println(c == ArrayList.class); // true: class literal

Object created = Student.class.getDeclaredConstructor(int.class, String.class)
                              .newInstance(1, "Dyn");   // create an object via reflection
// Class.newInstance() is deprecated (Java 9): use getDeclaredConstructor().newInstance()
```

NOTE

Frameworks such as Spring, Hibernate, JUnit and Jackson rely heavily on reflection: they discover annotations, call constructors and set fields at runtime.

8.6 clone(): Shallow vs Deep Copy

`Object.clone()` creates a field-by-field copy, but only if the class implements the `Cloneable` marker interface; otherwise it throws `CloneNotSupportedException` (a checked exception). `clone()` is `protected` in `Object`, so you override it as `public` and can use a **covariant** return type.

```
class Address {
    String city;
    Address(String city) { this.city = city; }
}

class Customer implements Cloneable {
    String name;
    Address address;                                  // reference field
    Customer(String n, Address a) { name = n; address = a; }

    @Override
    public Customer clone() {                         // public + covariant return
        try {
            return (Customer) super.clone();          // SHALLOW copy
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);              // can't happen: we are Cloneable
        }
    }

    public Customer deepClone() {
        Customer copy = clone();
        copy.address = new Address(address.city);     // DEEP: copy nested objects too
        return copy;
    }
}

Customer c1 = new Customer("Rohit", new Address("Pune"));
Customer shallow = c1.clone();
shallow.address.city = "Mumbai";
System.out.println(c1.address.city);       // Mumbai  <- shared Address changed!

Customer c2 = new Customer("Amit", new Address("Delhi"));
Customer deep = c2.deepClone();
deep.address.city = "Chennai";
System.out.println(c2.address.city);       // Delhi   <- independent copy
```

```
SHALLOW:  c1 ----> [Customer name="Rohit", address --+]
                                                      +--> [Address city]  (ONE shared object)
          shallow -> [Customer name="Rohit", address --+]

DEEP:     c2 ----> [Customer, address ---> [Address "Delhi"]]
          deep --> [Customer, address ---> [Address "Chennai"]]   (separate objects)
```

	Shallow copy	Deep copy
Primitive fields	Copied	Copied
Immutable references (String)	Shared (safe)	Shared (safe)
Mutable references (Address, List, arrays)	**Shared**, so changes leak	Copied recursively, independent
Cost	Cheap	More work and memory

> **TIP**
>
> `clone()` is widely considered a flawed API: a checked exception, a marker interface, no constructor call, and trouble with final fields. In modern code prefer a **copy constructor** (`new Customer(other)`), a **static factory** (`Customer.copyOf(other)`), or **immutable objects** that never need copying. Arrays are the exception: `arr.clone()` is fine.

8.7 finalize() and Resource Cleanup

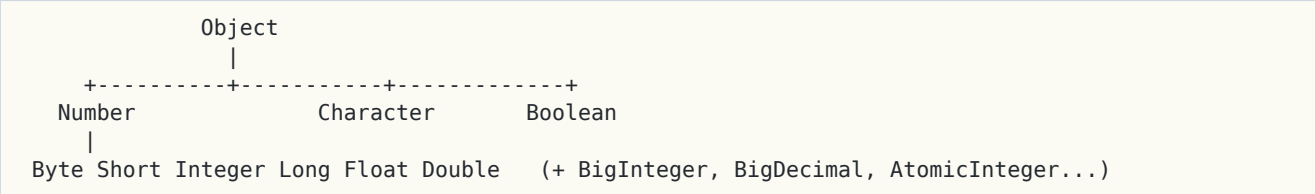
`finalize()` was meant to run before an object is garbage-collected. It is **unpredictable** (it might never run), slow and error-prone. It was deprecated in Java 9 and marked **for removal** in Java 18. Use instead:

- **try-with-resources** with `AutoCloseable` for files, sockets and DB connections (Phase 9).
- `java.lang.ref.Cleaner` (Java 9) for rare native-resource safety nets.

8.8 Wrapper Classes

Each primitive has a **wrapper class** in `java.lang` that represents the value as an **object**. All wrappers are **final** and **immutable**.

Primitive	Wrapper	Parent	Notable statics
byte	Byte	Number	parseByte, MIN_VALUE/MAX_VALUE
short	Short	Number	parseShort
int	Integer	Number	parseInt, valueOf, toBinaryString, toHexString, sum, max, compare
long	Long	Number	parseLong
float	Float	Number	parseFloat, isNaN
double	Double	Number	parseDouble, isNaN, isInfinite
char	Character	Object	isDigit, isLetter, toUpperCase, isWhitespace
boolean	Boolean	Object	parseBoolean, TRUE/FALSE



Why wrappers?

1. **Collections and generics work only with objects**: `List<Integer>`, not `List<int>`.
2. They can be `null`, which represents "no value" (for example a nullable database column).
3. They provide **utility methods**: parsing, conversion, comparison, constants.
4. Some APIs need objects: synchronization, reflection, serialization.

Boxing and unboxing

```
int i = 10;
Integer boxed = Integer.valueOf(i);      // boxing (explicit)
Integer auto = i;                        // AUTOBOXING (Java 5): Integer.valueOf(i)
int back = auto;                         // AUTO-UNBOXING: auto.intValue()

Integer a = 20, b = 30;
Integer sum = a + b;                     // unbox, add as ints, box again -> 50

List<Integer> list = new ArrayList<>();
list.add(5);                             // autoboxing
int first = list.get(0);                 // auto-unboxing
```

> **CORRECTED FROM ORIGINAL NOTES**
> The original notes create wrappers with constructors such as `new Integer(10)`, `new Byte(b)` and `new Character('A')`. These constructors are **deprecated since Java 9** and **marked for removal since Java 16**. Use `Integer.valueOf(10)`, autoboxing (`Integer x = 10;`) or `Integer.parseInt("10")`. Also, `Integer` implements `Comparable<Integer>`, not `Comparator<Integer>` as written in the original notes.

The Integer cache and the == trap

`Integer.valueOf()` (and therefore autoboxing) **caches** values from **-128 to 127**, so equal small values share the same object. Outside that range, new objects are created. The same caching applies to `Byte`, `Short`, `Long`, `Character` (0–127) and `Boolean`.

```
Integer x1 = 127, x2 = 127;
System.out.println(x1 == x2);        // true   (same cached object)

Integer y1 = 128, y2 = 128;
System.out.println(y1 == y2);        // false  (two different objects!)
System.out.println(y1.equals(y2));   // true   (compare VALUES with equals)
System.out.println(y1.intValue() == y2);   // true (one side unboxed -> numeric compare)
```

PITFALL

Never compare wrapper objects with `==`. Use `equals()`, `Integer.compare()`, or unbox. Tests that pass with small IDs and fail with large ones are a classic production bug.

Unboxing null throws NullPointerException

```
Map<String, Integer> stock = new HashMap<>();
int qty = stock.get("APPLE");     // NullPointerException: get() returned null -> unboxing
int safe = stock.getOrDefault("APPLE", 0);   // OK

Integer maybe = null;
if (maybe == 5) { }               // NullPointerException as well
```

Useful wrapper methods

Method	Example	Result
`Integer.parseInt(s)` / `parseInt(s, radix)`	`Integer.parseInt("ff", 16)`	`255` (primitive)
`Integer.valueOf(s)`	`Integer.valueOf("42")`	`Integer` object
`xxxValue()`	`Integer.valueOf(7).doubleValue()`	`7.0`
`Integer.toBinaryString` / `toHexString` / `toOctalString`	`Integer.toBinaryString(10)`	`"1010"`
`Integer.compare(a, b)`	`Integer.compare(3, 7)`	negative
`Integer.sum`, `max`, `min`	`Integer.max(3, 7)`	`7` (handy as method references)
`Integer.bitCount(n)`	`Integer.bitCount(7)`	`3`
`Character.isDigit / isLetter / isLetterOrDigit`	`Character.isDigit('7')`	`true`
`Character.getNumericValue(c)`	`Character.getNumericValue('7')`	`7`
`Double.isNaN(d)` / `Double.compare(a, b)`	`Double.isNaN(0.0 / 0)`	`true`
`Boolean.parseBoolean(s)`	`Boolean.parseBoolean("TRUE")`	`true` (anything else gives false)

TIP

Boxing has a cost. Summing a `Long` in a loop (`Long total = 0L; total += x;`) creates millions of objects. Use primitives for heavy arithmetic, and primitive streams (`IntStream`, `mapToInt`) in stream pipelines.

8.9 The Math Class

`java.lang.Math` is a `final` utility class with a private constructor and only **static** methods and constants (`Math.PI`, `Math.E`).

Method	Example	Result	Note
`abs(x)`	`Math.abs(-7.5)`	`7.5`	`Math.abs(Integer.MIN_VALUE)` is still **negative** (overflow)
`max(a, b)` / `min(a, b)`	`Math.max(10, 20)`	`20`	
`pow(a, b)`	`Math.pow(2, 10)`	`1024.0`	Always returns `double`
`sqrt(x)` / `cbrt(x)`	`Math.sqrt(25)`	`5.0`	`sqrt(-1)` is `NaN`
`ceil(x)`	`Math.ceil(4.1)`	`5.0`	Rounds up
`floor(x)`	`Math.floor(4.9)`	`4.0`	Rounds down
`round(x)`	`Math.round(4.5)`	`5` (long)	Half rounds **up** toward +∞: `round(-4.5)` is `-4`
`rint(x)`	`Math.rint(4.5)`	`4.0`	Half rounds to **even**
`random()`	`Math.random()`	`[0.0, 1.0)`	See below
`floorDiv` / `floorMod`	`Math.floorMod(-7, 3)`	`2`	vs `-7 % 3` gives `-1`
`addExact`, `subtractExact`, `multiplyExact`, `toIntExact`	`Math.addExact(Integer.MAX_VALUE, 1)`	`ArithmeticException`	Overflow-safe arithmetic
`hypot(x, y)`	`Math.hypot(3, 4)`	`5.0`	√(x² + y²)
`signum(x)`	`Math.signum(-3.2)`	`-1.0`	
`sin`, `cos`, `tan`, `log`, `log10`, `exp`	`Math.sin(Math.toRadians(30))`	`≈ 0.5`	Trig functions take **radians**
`clamp(v, min, max)` (Java 21)	`Math.clamp(150, 0, 100)`	`100`	

```java
// Random integer in [min, max] inclusive
int dice = (int) (Math.random() * 6) + 1;                    // 1..6
int r = ThreadLocalRandom.current().nextInt(100, 1000);      // 100..999 (preferred)

// Security-sensitive values (OTP, tokens, salts): use SecureRandom
SecureRandom sr = new SecureRandom();
String otp = String.format("%06d", sr.nextInt(1_000_000));   // e.g. "048213"

import static java.lang.Math.*;                              // static import
double area = PI * pow(5, 2);                                // no "Math." prefix
```

PITFALL

Don't use `Math.random()` or `java.util.Random` for OTPs, passwords or tokens, because they are predictable. Use `SecureRandom`.

8.10 BigDecimal and BigInteger

`BigInteger` holds integers of **arbitrary size** (factorials, cryptography). `BigDecimal` holds **exact decimal** numbers with controllable scale and rounding, which makes it **the** type for money in banking and fintech.

```
import java.math.*;

BigDecimal principal = new BigDecimal("100000.00");      // use the String constructor
BigDecimal rate = new BigDecimal("0.075");
BigDecimal interest = principal.multiply(rate)
                               .setScale(2, RoundingMode.HALF_UP);    // 7500.00
BigDecimal total = principal.add(interest);                           // 107500.00

new BigDecimal(0.1);         // 0.1000000000000000055511151231257827... (inexact!)
BigDecimal.valueOf(0.1);     // 0.1 (uses Double.toString, which is safe)

BigDecimal emi = total.divide(new BigDecimal("12"), 2, RoundingMode.HALF_EVEN);  // 8958.33
// total.divide(new BigDecimal("3"));   // ArithmeticException: non-terminating decimal

BigInteger big = BigInteger.valueOf(2).pow(100);   // 1267650600228229401496703205376
```

Rule	Why
Create from a `String` (or `BigDecimal.valueOf(double)`)	`new BigDecimal(0.1)` captures the binary error of the double
Always pass a scale and `RoundingMode` to `divide`	Otherwise non-terminating results throw
`HALF_EVEN` (banker's rounding) / `HALF_UP`	Choose according to your financial rules
Compare with `compareTo`, not `equals`	`equals` also compares scale (`2.0` ≠ `2.00`)
BigDecimal is **immutable**	`a.add(b)` returns a new value, so assign it

8.11 Immutable Classes

An **immutable** object's state can't change after construction. Examples in the JDK: `String`, all wrappers, `BigDecimal`, `BigInteger`, `LocalDate`/`LocalDateTime`, `List.of(...)` and records.

Rules for writing one

Rule	Why
Declare the class `final` (or give it private constructors)	Subclasses can't add mutable behaviour
Make all fields `private final`	Assigned once, not visible outside
Provide **no setters** or other mutators	Nothing can change state
Initialise everything in the constructor	The object is fully built from the start
Defensive-copy mutable inputs in the constructor	The caller can't change your internals through its own reference
Return copies or unmodifiable views of mutable fields from getters	Callers can't change your internals through the returned reference
"Modifications" return **new** objects (`withX(...)`)	Like `String.concat` and `LocalDate.plusDays`

```java
import java.time.LocalDate;
import java.util.List;

public final class Transaction {
    private final String txnId;
    private final BigDecimal amount;          // immutable type: safe to share
    private final LocalDate date;             // immutable type: safe to share
    private final List<String> tags;          // mutable type: needs defensive copying

    public Transaction(String txnId, BigDecimal amount, LocalDate date, List<String> tags) {
        this.txnId = txnId;
        this.amount = amount;
        this.date = date;
        this.tags = List.copyOf(tags);        // defensive, unmodifiable copy (Java 10)
    }

    public String getTxnId()      { return txnId; }
    public BigDecimal getAmount() { return amount; }
    public LocalDate getDate()    { return date; }
    public List<String> getTags() { return tags; }   // already unmodifiable

    public Transaction withAmount(BigDecimal newAmount) {   // "wither" returns a NEW object
        return new Transaction(txnId, newAmount, date, tags);
    }
}
```

PITFALL

Without the defensive copy, a caller can still change your "immutable" object: `List<String> t = new ArrayList<>(); var tx = new Transaction(..., t); t.add("HACK");` changes `tx`'s tags. Old mutable types such as `java.util.Date` need copying too, which is one reason to prefer `java.time`.

Benefits

- **Thread-safe** without synchronization: safe to share between threads.
- **Safe as `HashMap` keys and `HashSet` elements**: the hash never changes.
- **Cacheable and reusable** (like the String pool and the Integer cache).
- **Easier to reason about and test**: no hidden state changes.
- The trade-off is that every change creates a new object (usually negligible).

MODERN JAVA

A **record** (Java 16) is a concise immutable data carrier: `public record Money(BigDecimal amount, String currency) { }`. It generates the private final fields, constructor, accessors, `equals`, `hashCode` and `toString`. Add validation and defensive copies in a *compact constructor*. Records are covered in Phase 15.

8.12 Predict the Output

```
Integer a = 100, b = 100, c = 1000, d = 1000;
System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));   // true false true

Long l = 127L;
System.out.println(l.equals(127));        // false (Integer vs Long: different types)
System.out.println(l == 127);             // true  (unboxed numeric comparison)

System.out.println(Math.round(-2.5) + " " + Math.round(2.5));        // -2 3
System.out.println(Math.floorMod(-7, 3) + " " + (-7 % 3));           // 2 -1
System.out.println(Math.abs(Integer.MIN_VALUE));                     // -2147483648

System.out.println(new BigDecimal("2.0").equals(new BigDecimal("2.00")));       // false
System.out.println(new BigDecimal("2.0").compareTo(new BigDecimal("2.00")));    // 0

Object o = new Object();
System.out.println(o.equals(o) + " " + o.equals(null));             // true false

Character ch = 'A';
System.out.println(ch + 1);               // 66 (unboxed to char, promoted to int)
```

8.13 Interview Questions

Q. Why must `equals()` and `hashCode()` be overridden together?

Hash-based collections locate the bucket by `hashCode()` and then confirm with `equals()`. If equal objects have different hash codes, they land in different buckets, so duplicates appear in a `HashSet` and `HashMap.get()` misses entries.

Q. Can two unequal objects have the same hash code?

Yes, that is a collision, and it's allowed. The reverse isn't: equal objects must have equal hash codes.

Q. What is the default `toString()` output?

`getClass().getName() + "@" + Integer.toHexString(hashCode())`, for example `Student@1b6d3586`.

Q. What is the difference between a shallow and a deep copy?

A shallow copy copies field values, so referenced objects are shared. A deep copy also copies the referenced mutable objects, so the copy is fully independent. `Object.clone()` is shallow by default.

Q. Why is `Cloneable` called a marker interface? What if you call `clone()` without it?

It has no methods; it only signals that `Object.clone()` is permitted. Without it, `super.clone()` throws `CloneNotSupportedException`.

Q. What are wrapper classes and why are they needed?

Object representations of the eight primitives. They are needed for collections and generics, for nullability, and for utility methods. They are final and immutable.

Q. What is autoboxing? Where can it cause problems?

Automatic conversion between primitives and wrappers (`valueOf` and `xxxValue` inserted by the compiler). Problems: `==` comparisons outside the -128..127 cache, `NullPointerException` when unboxing `null`, and performance cost in loops.

Q. Why does `Integer a = 127, b = 127; a == b` give true but 128 gives false?

`Integer.valueOf` caches -128 to 127, so 127 returns the same object. 128 creates distinct objects, so `==` (a reference comparison) is false.

Q. Why use BigDecimal instead of double for money?

`double` is binary floating point and can't represent many decimal fractions exactly, so rounding errors accumulate. `BigDecimal` stores exact decimals with explicit scale and rounding modes.

Q. How do you create an immutable class?

Make it final; make all fields private final; add no setters; initialise everything in the constructor; defensively copy mutable inputs and outputs; return new objects for changes. Or use a record (with defensive copies for mutable components).

Q. What is the difference between `Math.round`, `Math.ceil`, `Math.floor` and `Math.rint`?

`ceil` rounds up, `floor` rounds down, `round` rounds half toward positive infinity (and returns a long/int), and `rint` rounds half to even (and returns a double).

8.14 Quick Revision

- `Object` has 11 methods. `getClass`, `wait` and `notify`/`notifyAll` are final; `clone` is protected; `finalize` is deprecated.
- Override `toString()` for readable logs, but never print secrets.
- `equals` contract: reflexive, symmetric, transitive, consistent, `x.equals(null)` is false. Check `this == o` and `instanceof` first.
- **Equal objects ⇒ equal hash codes.** Override both with the **same fields**; `Objects.hash(...)` helps.
- Don't mutate fields of objects inside hash-based collections.
- `clone()` needs `Cloneable` and is shallow by default; prefer copy constructors or factories; deep copy nested mutables.
- Wrappers: final, immutable. `Number` is the parent of the numeric ones. Use `valueOf`, not the deprecated constructors.
- Autoboxing uses `valueOf`; unboxing uses `xxxValue` → NPE on `null`.
- Integer cache: -128..127. **Never compare wrappers with `==`.**
- Math: static utilities. `round(-2.5)` is -2; `floorMod` for positive modulo; `*Exact` methods for overflow safety; `SecureRandom` for OTPs.
- Money: `BigDecimal` from a String, explicit `RoundingMode`, compare with `compareTo`.
- Immutable class: final class, private final fields, no setters, defensive copies, withers. Records simplify this.